ForecastLabAI — modular monolith architecture

Problem

Describe what sits behind the model and defend the "one FastAPI app" choice.

Instant solution

Shape: modular monolith — 19 feature modules in `app/features/`, one FastAPI process, one PostgreSQL + pgvector database, external LLM providers.

Module	Responsibility
`data_platform`, `dimensions`	retail entities: stores, products, sales, price, promo, inventory
`ingest`	validate and upsert incoming sales data
`featuresets`	time-safe feature engineering
`forecasting`	train models, generate forecasts
`backtesting`	chronological fold evaluation
`model_selection`	rank candidates, pick a champion
`registry`	runs, metrics, artifacts, deployment aliases
`jobs`, `batch`	job tracking; batch = portfolio fan-out run sequentially via JobService
`rag`	index docs, semantic retrieval
`agents`	agent sessions, tools, WebSocket streaming, approvals
`scenarios`	baseline vs what-if comparison
`explainability`, `analytics`	forecast explanations, KPIs and series
`seeder` ("The Forge")	reproducible synthetic retail data
`config`, `ops`, `demo`	runtime API keys, read-only ops control center, showcase pipeline

Decisions to defend

- **Modular monolith:** clear boundaries, one deploy. Microservices only when parts need independent scaling (e.g. training workers) or teams.
- **JSONB** for run configs/metrics: model-specific params without a schema per model.
- **Deployment aliases:** stable logical name → specific run; swap the champion without changing callers.
- **SHA-256 artifact hash:** checked on load (`registry/storage.py`, `ChecksumMismatchError`). Integrity check only, not proof an artifact is safe to deserialize.
- **Pydantic everywhere:** one contract style for API requests, services, agent tools and agent outputs.
- **WebSocket** (`/agents/stream`): streamed agent replies instead of waiting for the full answer.

Why it works / gotchas

- There is **no authentication layer**. API keys are managed at runtime (`features/config`) and never returned or logged.
- What-if re-forecasting with features (`model_exogenous`) only works for the regression model; all other models get a labelled heuristic. Estimates, not causal effects.

Diagram brief

- Type: component diagram.
- Top: React dashboard (Explorer · Forecasts · Demand Planner · What-If · Agent Chat) — edge "REST / WebSocket" → FastAPI box.
- Inside FastAPI: module boxes grouped as Data (data_platform, ingest, featuresets) · ML (forecasting, backtesting, model_selection, registry, jobs/batch) · GenAI (rag, agents) · Decision (scenarios, explainability, analytics).
- Bottom: PostgreSQL + pgvector (sales · runs · jobs · embeddings · sessions). Right: LLM providers (OpenAI · Anthropic · optional Ollama).
- Do NOT draw separate services/containers per module.